# Plano de aula e material de apoio aula ao vivo

## Aula 02

### Do diagrama ao código em JavaScript

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Objetivo da Aula e Capacidade

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

## ORIENTAÇÕES PEDAGÓGICAS

### OBJETIVO DA AULA

Implementar em JavaScript o modelo de classes do cadastro de clientes da Electronic, aplicando herança, encapsulamento e validação de valores não permitidos.

### CAPACIDADES

**Básica:** aplicar as características do paradigma Orientado a Objetos (POO) no desenvolvimento de aplicativos.

**Socioemocional:** constatar a iniciativa como requisito fundamental para uma postura inovadora e aberta a novas experiências e aprendizados.

# Conhecimentos Relacionados

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

## ORIENTAÇÕES PEDAGÓGICAS

### POR QUE ESTES CONHECIMENTOS?

Eles permitem transformar o diagrama de classes da Atividade em um programa que cadastra pessoas físicas e jurídicas e recusa valores inválidos.

### CONHECIMENTOS

- Linguagens orientadas a objetos adotadas pelo mercado
- Técnicas de programação: sintaxe, variáveis, decisão e repetição
- Classes, construtores e herança em JavaScript
- Encapsulamento: campos privados, getters e setters
- Tratamento de erros com try e catch
- Versionamento com Git e GitHub

# Recursos e Ambientes Pedagógicos

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

**ORIENTAÇÕES PEDAGÓGICAS**

**01**

## Visual Studio Code

Editor gratuito para escrever e organizar o código do projeto.

**02**

## Node.js

Ambiente para executar JavaScript fora do navegador, pelo terminal.

**03**

## Git e GitHub

Versionamento e publicação do projeto (git-scm.com e github.com).

**04**

## AVA

Entrega do link do repositório e fórum de dúvidas da unidade.

# Roteiro de Estudos e Avaliações

ORIENTAÇÕES PEDAGÓGICAS

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

| Semana 2 | |
|---|---|
| **Estudar** | **Livro Didático**<br>Desafio 2 – Linguagem de programação orientada a objetos, pág. 30 a 55.<br>Material complementar: instalação do IntelliJ e do Dart SDK e técnicas de programação, pág. 58 a 97.<br><br>**Material Online**<br>Aula 2 (Desafio 2): Linguagem de programação orientada a objetos (leitura de todas as telas)<br>PDFs de apoio: Instalando o IntelliJ, Instalando o Dart e Técnicas de programação |
| **Participar** | Aula ao vivo 2: Do diagrama ao código em JavaScript<br>Fórum de dúvidas |
| **Responder** | **Atividade de Entrega 01: Cadastro de Clientes da Electronic**<br>**Recuperação da Avaliação Online 1** |

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Agenda da aula

| Momento | Tempo | Conteúdo |
|---|---|---|
| **1. Revisão** | 15 min | Pontos de atenção nos diagramas de classes da Atividade 2 |
| **2. Do diagrama ao código** | 75 min | Classes, herança, encapsulamento e validação, com demonstração ao vivo |
| **3. Ambiente e GitHub** | 20 min | VS Code, Node.js, Git e publicação do repositório |
| **4. Atividade 3** | 10 min | Lançamento, critérios de avaliação e tira-dúvidas |

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Revisão da modelagem

- A seta de herança aponta da classe derivada para a classe base.
- O que é comum a pessoa física e jurídica fica na classe base; o específico fica em cada derivada.
- Os atributos têm visibilidade definida e são acessados por métodos.
- O endereço está presente nos dois cadastros.
- Nomes de classes, atributos e métodos já pensados para virar código.

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Do diagrama ao código

| No diagrama UML | Em JavaScript | Exemplo |
|---|---|---|
| **Classe** | class | class Pessoa { } |
| **Atributo privado (-)** | campo privado com # | #nome; |
| **Herança (seta)** | extends e super() | class PessoaFisica extends Pessoa |
| **Acesso ao atributo** | get e set | get nome() e set nome(valor) |

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Endereço e validações

## Função reaproveitada pelas classes e a classe Endereco

```
const apenasDigitos = (valor) => String(valor).replace(/\D/g, "");

function validarTamanho(valor, tamanho, rotulo) {
  const digitos = apenasDigitos(valor);
  if (digitos.length !== tamanho) {
    throw new Error(`${rotulo} deve ter ${tamanho} dígitos`);
  }
  return digitos;
}

class Endereco {
  constructor(logradouro, numero, cidade, uf) {
    this.logradouro = logradouro;
    this.numero = numero;
    this.cidade = cidade;
    this.uf = uf;
  }

  toString() { return `${this.logradouro}, ${this.numero} - ${this.cidade}/${this.uf}`; }
}
```

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Classe base: Pessoa

**#nome é privado; o acesso passa por get e set, e o setter valida**

```
class Pessoa {
  #nome;
  endereco;

  constructor(nome, endereco) {
    this.nome = nome;           // passa pelo setter e valida
    this.endereco = endereco;
  }

  get nome() { return this.#nome; }

  set nome(valor) {
    if (typeof valor !== "string" || valor.trim().length < 3)
      throw new Error("Nome deve ter ao menos 3 caracteres");
    this.#nome = valor.trim();
  }

  toString() { return `${this.nome} | ${this.endereco}`; }
}
```

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# PessoaFisica: herança

**extends herda de Pessoa; super() chama o construtor da classe base**

```
class PessoaFisica extends Pessoa {
  #cpf;
  #dataNascimento;
  constructor(nome, endereco, cpf, dataNascimento) {
    super(nome, endereco);       // reaproveita a classe base
    this.cpf = cpf;
    this.dataNascimento = dataNascimento;
  }

  get cpf() { return this.#cpf; }
  set cpf(valor) { this.#cpf = validarTamanho(valor, 11, "CPF"); }
  get dataNascimento() { return this.#dataNascimento; }
  set dataNascimento(valor) {
    const data = new Date(valor);
    if (Number.isNaN(data.getTime()) || data > new Date())
      throw new Error("Data de nascimento inválida");
    this.#dataNascimento = data;
  }

  toString() { return `${super.toString()} | CPF: ${this.cpf}`; }
}
```

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# PessoaJuridica: herança

## Mesma base, atributos específicos: CNPJ e razão social

```
class PessoaJuridica extends Pessoa {
  #cnpj;
  #razaoSocial;
  constructor(nome, endereco, cnpj, razaoSocial) {
    super(nome, endereco);
    this.cnpj = cnpj;
    this.razaoSocial = razaoSocial;
  }

  get cnpj() { return this.#cnpj; }
  set cnpj(valor) { this.#cnpj = validarTamanho(valor, 14, "CNPJ"); }

  get razaoSocial() { return this.#razaoSocial; }
  set razaoSocial(valor) {
    if (!valor || valor.trim() === "")
      throw new Error("Razão social é obrigatória");
    this.#razaoSocial = valor.trim();
  }

  toString() { return `${super.toString()} | CNPJ: ${this.cnpj}`; }
}
```

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Usando as classes

## try e catch tratam os valores não permitidos sem derrubar o programa

```
const endereco = new Endereco("Rua das Flores", 120, "Fortaleza", "CE");
const clientes = [];

try {
  clientes.push(new PessoaFisica("Ana Souza", endereco, "123.456.789-09", "1990-05-10"));
  clientes.push(new PessoaJuridica("Loja Tech", endereco, "12.345.678/0001-95", "Loja Tech Ltda"));
  clientes.push(new PessoaFisica("Jo", endereco, "123", "2050-01-01"));   // inválido
} catch (erro) {
  console.error(`Erro: ${erro.message}`);
}

clientes.forEach((cliente) => console.log(String(cliente)));
```

### SAÍDA NO TERMINAL

```
Erro: Nome deve ter ao menos 3 caracteres
Ana Souza | Rua das Flores, 120 - Fortaleza/CE | CPF: 12345678909
Loja Tech | Rua das Flores, 120 - Fortaleza/CE | CNPJ: 12345678000195
```

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Ambiente e GitHub

## Comandos para executar o projeto e publicá-lo no GitHub

```
# 1. Conferir se as ferramentas estão instaladas
node -v
git --version

# 2. Criar a pasta do projeto e iniciar o repositório
mkdir cadastro-electronic
cd cadastro-electronic
git init

# 3. Executar o código
node index.js

# 4. Versionar e publicar no GitHub
git add .
git commit -m "Implementa o cadastro de clientes da Electronic"
git branch -M main
git remote add origin https://github.com/SEU-USUARIO/cadastro-electronic.git
git push -u origin main
```

# Critérios da Atividade

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

**01**

### Ambiente

Preparou corretamente o ambiente de desenvolvimento.

**02**

### Classes base

Criou as classes base de pessoa física e jurídica com nome e endereço.

**03**

### Pessoa física

Criou os métodos de acesso a CPF e data de nascimento.

**04**

### Pessoa jurídica

Criou os métodos de acesso a CNPJ e razão social.

**05**

### Código

Escreveu o código com semântica e sintaxe adequadas.

**06**

### Prazo

Entregou a atividade no prazo estabelecido.

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Entrega e feedback

- ➔ Publique o projeto no GitHub e compartilhe o link no AVA.
- ➔ A entrega vale 50% da nota e deve respeitar o prazo.
- ➔ O feedback pode ser dado na hora, ao concluir a atividade, ou online no AVA.
- ➔ Dúvidas durante a atividade: use o fórum de dúvidas do AVA;

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

# Encerramento

- Classe, herança e encapsulamento: class, extends e super, campos privados com # e get e set.
- Valores não permitidos: o setter lança um erro e o try e catch trata a falha.
- Entrega: repositório no GitHub e link no AVA.
- Próximos passos: desenvolver a Atividade e usar o fórum de dúvidas sempre que precisar.

# Obrigado!

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA